

← Software Engineering Advising Project Report

Names: Sajjad Shamoosi, Sebastian Chapa, Justus Polston, Landon Simms, Eduardo Marquez, Jose Balderas-Soto

Project Team: Advisor Assembly

Page of 10

Preface: This document is created for software engineers to reference in future parts of the process and for users to use to verify that their requests and preferences have been taken into consideration.

Introduction: Currently, advising at the university is limited by the number of advisors and their office hours. It may take students large or unreasonable amounts of time to set up a meeting, even for simple needs. As such, this system was created to address these concerns by creating a system that can handle small, simple, and common requests, such as creating a schedule for the next semester or taking classes in an optimal order. The system should interact with external databases that contain up to date user log in information, particularly the SSO/Single Sign On (TXST).

Glossary:

- **Database** - An organized collection of data/information designed for efficient storage and retrieval (GeeksforGeeks)
 - **External Database** - A database that exists outside of the main system infrastructure (ITU Online)
- **Prerequisites** - Something that must be completed as a requirement for something else
- **Functional Requirement** - Statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations (Sommerville).
- **Nonfunctional Requirement** - Constraints on the services or functions offered by the system, including timing constraints, constraints on the development process, and constraints imposed by standards (Sommerville).

I. Requirements

User Requirements:

- **Functional**
 - 2.1.1: Users shall have the ability to log in with school affiliated emails and passwords
 - 2.1.2: Users shall be able to input information about their majors, minors, completed classes, and credit hours completed
 - 2.1.3: Users shall be able to generate a list of classes necessary for completing their degree
 - 2.1.4: Users shall be able to generate a sample plan for upcoming semesters
- **Nonfunctional**
 - 2.2.1: The system should retrieve user information from the school's database to authenticate user logins
 - 2.2.2: The system should be available for use 24 hours a day, 7 days a week
 - 2.2.3: The system should have a capacity of 99999 maximum simultaneous users

← Software Engineering Advising Project Report

System Requirements:

- **Functional**

- 3.1.1.1: The system shall accept user input with their login information
- 3.1.1.2: The system shall search the school's database to locate potential matches
- 3.1.1.3: Upon successful verification, the system shall provide access and generate the user interface
- 3.1.2.1: The system shall request user input regarding majors, minors, credit hours completed, and classes completed
- 3.1.2.2: The system shall accept user input that fit those criteria, storing the data for future reference
- 3.1.3.1: The system shall generate a list of classes the user requires for degree completion
- 3.1.3.2: The system shall sort the list of classes according to their priority and update the user interface with the result
- 3.1.4.1: The system shall utilize user provided data to generate a sample schedule with classes that meet their specified criteria
- 3.1.4.2: The system shall disregard classes that the user has already received credits for when generating a schedule
- 3.1.4.3: The system shall avoid classes with overlapping timeframes when generating a schedule

- **Nonfunctional**

- 3.2.1.1: The system should reject user authentication requests if a match cannot be found in the school's database within 15 seconds
- 3.2.1.2: The system should generate a user interface with access to system functions within 5 seconds of successful authentication
- 3.2.2.1: The system should be able to store user data for up to 999999 users
- 3.2.2.2: The system should clear a user's stored data after 12 months have passed since they last accessed the system
- 3.2.3.1: The system should be accessible and operable at any time or date, unless it is under maintenance
- 3.2.3.2: Time spent on system maintenance should not exceed 15 minutes in any single day
- 3.2.4: The system should be able to accommodate and provide services for up to 99999 active users

Requirements Version History:

- **Version 0**

A list of requirements created through group brainstorming. These are unrefined, unsorted, and unorganized, but useful for future reference.

- **Version 1**

The requirements from V0 were further refined and expanded upon. Additionally, they have been properly organized and sorted. A preface, introduction, and glossary have been created to accompany them.

- **Version 2**

← Software Engineering Advising Project Report

The numbering system has been expanded upon to better relate various requirements to each other.

- Specifically:

- The first digit is 2 for user requirements and 3 for system requirements
- The second digit is 1 for functional and 2 for nonfunctional requirements
- The third digit is sequential and arbitrary, but shared third digits represent shared subject matters
- The fourth digit is an additional specification for requirements sharing the same third digit, representing related parts of a shared subject. These are also arbitrary and sequential.

Many requirements were changed, added, or removed based on current project needs and new understandings of related material.

Font size of the document has been increased to 13 to increase overall visibility.

- Version 4

Removed untestable requirements and cleaned up user numbers mentioned in 2.2.3, 3.2.2.1, and 3.2.4 to better fit partitioning tests. Maximum capacity 99,999 was chosen because there were 40,678 students enrolled at Texas State University as of Fall 2024 (TXST).

- Version 5

Updated requirement 2.2.3's grammar to be more consistent with other requirements.

- Version 6 (Current Version)

Updated requirement 2.1.2 and 3.1.2.1 to specify the specific user inputs they accept.

II. Architecture

Key:

Login/Auth: Present in all versions, this is for facilitating the login process into the system and communicating with the database for verification.

SSO: Present in all versions, stands for Single Sign-On. This represents the database used for TXST NetID login information (TXST).

Home: Present in version 1.0 - 2.0, this is for the homepage that the user is sent to after valid login. In version 3.0, it was transformed into a subsystem.

Class List: Present in all versions, this provides the functionality to generate the list of classes ranked by priority.

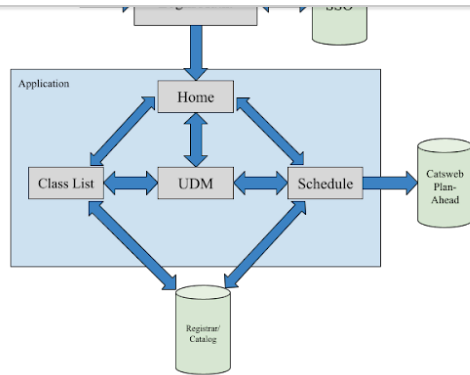
UDM: Present in all versions, stands for User Data Manager. This would temporarily store user input that can be fetched for use in other functions.

Schedule: Present in all versions, this generates a schedule for the next semester given user input and/or a generated class list.

Catsweb Plan-Ahead: Present in only version 1.0, this database represents where the data is stored for TXST's plan-ahead feature on catsweb (TXST).

Registrar/Catalog: Present in all versions, this is representing the database(s) that contains information about class requirements, class times, etc.

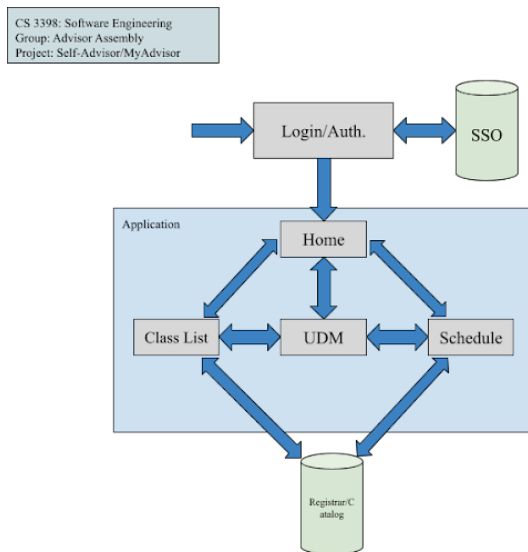
← Software Engineering Advising Project Report



Version History

Version 1.0:

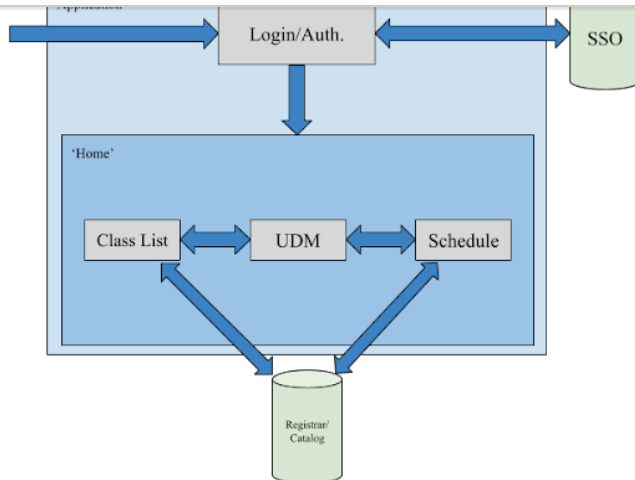
Initial architecture, drafted after initial prototypes and feedback.



Version 2.0:

Removed the Catsweb Plan-Ahead (TXST) database and the connection to it from Schedule. This was revised as it was outside our current scope of requirements.

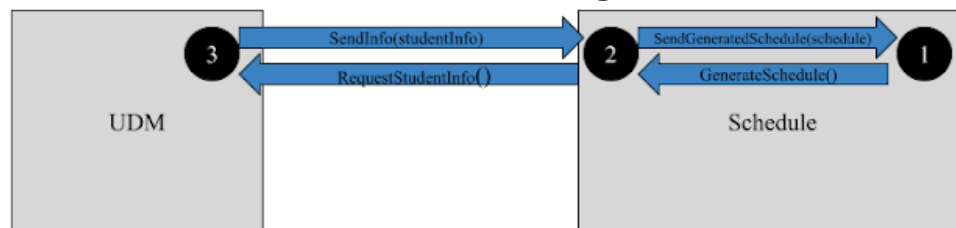
← Software Engineering Advising Project Report



Version 3.0:

Removed the home box, instead replacing it with a subsystem and restructuring the architecture accordingly. This was to better fit the functionality of home. Also moved the Login/Auth box into the Application to be more accurate.

III. Detail Design



Description: The above model shows three “bubbles” inside two “boxes”, representing processes and the system components they reside in respectively, and some of the interactions these processes (bubbles) have with each other. This model does not illustrate all processes, components, or interactions in the system; it is a simplified representation of a specific internal interaction between two processes inside the schedule component and an external interaction between the user data manager and schedule components.

Bubble/Process 1: Located on the top right of the model is a bubble labeled 1, representing process 1. This process would, in large scale implementation, be responsible for receiving and processing external requests. While the reception of such a request isn’t shown in the model, the model does show the result of this process receiving a proper request to generate a schedule. Once this request is received, it issues a **generateSchedule()** command to process 2 and accepts a completed schedule via the function **sendGeneratedSchedule(schedule)** in return.

Bubble/Process 2: Located near the model’s center and in the top left of the schedule component is a bubble labeled 2, representing process 2. This process is responsible for generating schedules, a task that it requires user information to complete. It receives a generation request from process 1, **generateSchedule()**, and requests information from

← Software Engineering Advising Project Report

sendGeneratedSchedule(schedule). Notably, the model shows this process performing internal and external interactions with process 1 and 3 respectively.

Bubble/Process 3: Located in the top left of the model is a bubble labeled 3, representing process 3. This process, located in the user data manager, would be responsible for handling external requests. While the internal interactions it would perform as a result of the interactions modeled above are not shown within the model, the external interaction it has with process 2 is detailed. Upon receiving the request, when process 2 calls **requestStudentInfo()**, it would call upon another process within the UDM to retrieve said information. Ultimately, it would then receive this information and send it back to process 2 with the function **sendInfo(studentInfo)**.

IV. Testing

Test Case #1, Requirement 2.1.1

- Name: User Login Success
- Description: Ensure that emails and passwords that exist in the database allow the user to login successfully
- Test Procedure:
 1. Initialize a dictionary containing userIDs and passwords to simulate a database
 - a. Will only contain the following key/value pair: {"stu01": "abc123"}
 2. Enter stu01 and abc123 in the login() function
 - a. Function parameters: login(userID, password)
- Results:
 - Pass - The user is successfully authorized
 - Fail - The user is not successfully authorized
- Remarks: Failure may be a result of exceptions, like an invalidStringException, incorrect usernames/password matches, or nonexistent usernames. As a result, this test failing is helpful for checking functionality but less useful for finding the cause of failure.

Test Case #2, Requirement 2.1.3

- Name: Course List Generation Failure
- Description: The system will generate a course list based on the information from the inputs
- Test Procedure:
 1. Set the user input to NULL
 2. Call the generateCourseList() function
- Results:
 - Pass - The system fails to generate a course list due to a lack of inputs
 - Fail - The system generates a course list despite the lack of inputs
- Remarks: The test is successful upon list generation failure, which can unfortunately be caused by a number of factors unrelated to a lack of user input. While the results of the test are useful, they're significantly more meaningful when list generation tests succeed in other scenarios as well.

Test Case #3, Requirement 2.2.3

← Software Engineering Advising Project Report

- Test Procedure:
 1. Set the user load to have 99,999 concurrent users
 2. Input the following user values: "Major: Education, Minor: Math, Credit Hours: 0, Classes Completed: 0."
 3. Generate a sample plan
- Results:
 - Pass - Return a sample plan containing the roadmap with classes a user would be expecting
 - Fail - A sample plan is not correctly generated
- Remarks: The number 99,999 is chosen as a value for partition testing (Sommerville) because it is the highest integer value for capacity, numbering 5 digits total, where the system is expected to function correctly. This test's failure may be due to system crashes, which may indicate issues with the system at this capacity, or sample plan output being invalid, which can potentially be unrelated.

Test Case #4, Requirement 2.2.3

- Name: System Runtime for No Users
- Description: Testing the system's functionality with zero active users on the service
- Test Procedure:
 1. Set the user load to 0 concurrent users
 2. Input the following values: "Major: Education, Minor: Math, Credit Hours: 0, Classes Completed: 0."
 3. Generate a sample plan
- Results:
 - Pass - The system does not return a valid sample plan
 - Fail - The system generates a valid sample plan
- Remarks: Similarly to test 3, the number 0 was chosen as a value for partition testing (Sommerville). It represents the smallest integer value, simply 1 digit, where the system is expected to not function. As this test evaluates the sample plan generation function, which is representative of overall system functionality for this test, it should be noted that a passing test may erroneously be a result of errors within the sample plan generation component rather than successful identification of capacity problems.

Test Case #5, Requirement 2.2.3

- Name: System Runtime with Over Max User Capacity
- Description: Testing the system's functionality with 100,000 active users on the service
- Test Procedure:
 1. Set the user load to have 100,000 concurrent users
 2. Input the following values: "Major: Education, Minor: Math, Credit Hours: 0, Classes Completed: 0."
 3. Run the course list generation
- Results:
 - Pass - The system fails to generate a course list
 - Fail - The system returns a valid course list

← Software Engineering Advising Project Report

numbering 6 digits. As with the similar test case 4, a successful test may be the result of a component failure (in this case, the list generation component) rather than overall system failure on account of excess capacity.

V. Sources

- Sommerville, Ian. *Software Engineering*. 10th ed., Pearson, 2015.
- TXST. "Catsweb: Students." *Catsweb*, www.catsweb.txst.edu/students.html.
- TXST. "Fall 2024 Enrollment Fact Sheet" *Texas State University*, <https://news.txst.edu/inside-txst/2024/fall-enrollment/fact-sheet.html>
- ITU Online, "What Is an External Database? - ITU Online IT Training." *ITU Online IT Training*, www.ituonline.com/tech-definitions/what-is-an-external-database/.
- GeeksforGeeks. "What Is Database?" *GeeksforGeeks*, 12 Sept. 2019, www.geeksforgeeks.org/dbms/what-is-database/.